

Final Project

Team Name: Game Classification CD

Team Members: Sara Cogoli and Hannah Dees

Project Description: Game Genre Classification

This project is a machine learning model that can detect and sort video game genres into categories such as, Action, RPG, Puzzle, Horror, Adventure, Sports, Simulation, Shooter, etc. The model will recognize keywords within the description or game tags to help identify genre. User will search for any keywords they like, and the model will give a list of games based on a similar genre and give descriptions. The main idea is for the user to be able to search for any game they would like, using simple keywords. For example, if the user searches for 'zombies', the model will respond with games under the Horror genre.

Research question:

How accurate will the machine model classify video games into genres using user-inputted text features, tags, and descriptions?

Project Status UPDATED:

(**milestone 1**) Ideas formed and created. Description thought out and wrote. Goals and objectives are identified. Kaggle dataset identified. We also decided we will be implementing BERT tokenizer, re, pandas, and Sklearn in our project. All have been imported and installed. We decided the model will use the title of the game, the tags, the description, and the genre of the games listed. GameSortProj.py file created to put the model into and pushed into repository. Final code started within the main project file. (**milestone 2**) Imported libraries needed, loaded the data set, started the user interaction search UI, started the BERT tokenizer code, started prediction mapping code, as well as started the train/test code for the model. (**milestone 3**) Continued adding code to the GameDataSet class, defining actions and attributes. Added functions to the prediction function. Added BertForSequenceClassifier-it's needed to predict genres (predicting labels). PyTorch imported at the top for handling the math and keeping the model moving forward. Adding torch also helps with tokenizing the imputed text by the user and prevents the model from storing unnecessary training information. (**milestone 4**) The GitHub README was updated with overview, goals, how it works, and key techniques used. I have updated the code to match the column names of the columns in the dataset that we are using. NOW that we have matching column names (I assumed what the column names were, and they were incorrect, so it was crashing) Now the model RUNS FULLY. Users can search for a game, and the model (using BERT) does its best prediction for the answer. THE MODEL IS UNTRAINED-training model being added next- functionality is all good. (**milestone 5**) Model is TRAINED, multiple rounds of training were completed. The first round of training contained 2 rounds of epoch- reading through the dataset. The outcome of the first train

was a 78% accuracy using 3000 data items from the set. The model behaved correctly, but genres were not matched correctly most of the time. The latest train contained 9000 items and had a 90% accuracy rate. When searching for specific genres, you are very likely to receive exactly what you typed, the more specific you get the less accurate it can be but overall is proofed for low-specific terms. FinalNotebook Jupyter notebook created oversighting all main categories of our project, including markdown headings to label each section and blocks of example code pulled directly from our python file. **(milestone 6)** Torch.save was implemented on the model and successfully saved! The model is saved in saved_model and in steam_bert_model The accuracy is now 93% using 12000 data items from the set. Completion of Project.

EDA:

Introduction: For our model, we are using a Kaggle Steam Game Dataset. The dataset contains more than 40k games from the steam shop with detailed data. The set contains 20 columns containing eighteen strings, one decimal, and one URL datatype.

URL GitHub: <https://github.com/hannahd049/Final-Project-CogoliDees.git>

Project Timeline

Milestone 1: Foundation (Weeks 4–5)

Finalize your project topic and research question

Acquire and inspect your dataset

Set up version control (GitHub repository)

Perform exploratory data analysis (EDA)

Create visualizations: distributions, correlations, class balance

Milestone 2: Data Preparation (Weeks 6–7)

Handle missing values (imputation or removal)

Encode categorical variables

Scale/normalize numerical features

Engineer new features based on domain knowledge

Create train/validation/test splits

Milestone 3: Baseline Model (Week 8)

Implement a simple model (logistic regression or decision tree)

Establish baseline metrics to improve upon

Document results as your benchmark

Milestone 4: Model Development (Weeks 10–12)

Choose one algorithm that fits your problem (e.g., Random Forest, XGBoost, or neural network)

Tune hyperparameters using cross-validation

Conduct error analysis: confusion matrix, misclassification patterns

Address challenges like class imbalance

Milestone 5: Model Refinement (Weeks 13–14)

Continue improving your chosen model

Finalize hyperparameter tuning

Document your model's strengths and limitations

Milestone 6: Documentation & Presentation (Weeks 15–16)

Write your project report

Create presentation slides (10–15 slides)

Practice your presentation (aim for 10–15 minutes)

Prepare for Q&A